

L4: 計算法学のための関数型プログラミング言語

——制定的規則を全域関数として、規制的契約を並行プロセスとして——

Wong Meng Weng*

L4 開発チーム

要旨

本稿では、計算法学のための静的型付き関数型プログラミング言語 *L4* と、その意味論的基盤、ならびに法分野の専門家が利用できるようにするための制御自然言語 (controlled natural language) 的な表層構文を提示する。*L4* は、法文に対して従来から区別されてきたが先行する形式体系がしばしば混同してきた二つの読みを明確に分離する。**制定的** (constitutive) 規則——何が何として数えられるか——は、Haskell 流の関数型プログラミングの系譜に連なる全域型付き関数として符号化され、型安全性、網羅的な場合分け、そしてあらゆる判断に対する監査可能な評価トレースをもたらす。**規制的** (regulative) 規則——誰が、いつまでに、何をなすべきか、さもなくば何が起こるか——は、その表示的意味が時刻付き事象トレースの集合となる義務論理的・時相論理的な層に符号化され、Hvitved の契約仕様言語 (CSL) から、そしてそれを通じて Hoare の通信逐次プロセス (CSP) と事象計算 (event calculus) から多くを借りている。同一のソースプログラムは、Mercury や Curry でおなじみの関数から関係への標準の変換を介して、第二の関係的な読みをも許す。これにより、*L4* は決定手続として前向きに評価され、仮説形成的計画のために後ろ向きに走らせ、さらに記号的モデル検査器へ遷移関係として書き出すことができ、法の抜け穴の探索は到達可能な「二重拘束」状態の探索となる。本稿では、この言語と、その実装 (構文解析器、型検査器、評価器、言語サーバ、REST/MCP 判断サービス、および複数のコンパイル基盤)、そして句読点を最小化した構文——括弧の代わりに字下げ、ditto 記号、句としての識別子——の設計理念を、非プログラマが法を黒白の文字で読み書きできるようにするという目標とともに述べる。

キーワード: 計算法学, rules as code, low-code, 制御自然言語, 関数型プログラミング, 義務論理, プロセス計算, 契約の形式化, 法的知識表現

1 序論

十分な距離をおいて眺めれば、企業とはその契約の総和である——株主、債権者、顧客、仕入先、従業員、そして国家との契約の。ところが、この最も根源的な企業資産の管理者である法務部門は、ワープロからほとんど進歩していない道具立ての中で仕事をしている。グラフィックデザイナーには Adobe のスイートが、プログラマには言語・コンパイラ・バージョン管理・デバッグ・テスト基盤の数々があるのに対し、弁護士には Microsoft Word しかない。これは際立った非対称である。な

* 計算法センター (Centre for Computational Law), シンガポール経営大学 (Singapore Management University), および Legalese. mengwong@legalese.com

ぜなら、両者の仕事はそれほど違わないからだ。プログラマも起草者も、もつれ合った事業上の目標を、多数の可動部品と敵対的な参加者からなる複雑なシステムの振る舞いを先取りしなければならない精密な文章的成果物へと翻訳しているのである。

本稿は、この溝を埋めるために構築されたプログラミング言語 L_4 について報告する。 L_4 は文書形式でも、条項ライブラリでも、契約ライフサイクル管理のデータベースでもない。それは、デスクトップ・パブリッシングという製品が現れる前に PostScript が言語であったのと同じ意味での、一つの言語である。その設計は、互いに相まって AI・法分野の既存研究と一線を描く三つの方針に立脚している。第一に、制定的規則と規制的規則という古典的区別 [50, 25] を真剣に受け止め、両者を単一の論理に押し込めるのではなく、それぞれにふさわしい形式的扱いを与える。第二に、その規制層を契約 DSL とプロセス計算に由来するトレースに基づく並行性を意識した意味論 [32, 28] に根ざさせ、多者間の義務・期限・救済 (reparation) を第一級の対象とする。第三に、それを意図的に制御自然言語 [36] として設計する。その具象構文は句読点を最小化し、括弧を字下げで置き換え、法文の句そのものを識別子として許すことで、法令とその形式化を並べて同一の文章だと認められるようにする。

■貢献 本稿の貢献は以下のとおりである。

- L_4 の設計と、その二層アーキテクチャ——制定的規則のための全域型付き関数型中核 (4 節) と規制的規則のための義務論理的・時相論理的層 (5 節) ——の論拠を述べる。
- 規制層の意味論的基盤をトレースに基づく表示的モデルとして与え、 L_4 の表層キーワード (HENCE, LEST, WITHIN, RAND, ROR) と、Hvitved の CSL および Hoare の CSP の演算子との対応を示す (5 節)。
- 単一の L_4 プログラムが、Mercury や Curry で用いられる「関数のグラフ」変換を介して関数的・関係的の二つの読みを支えること、そして関係的読みが仮説形成的計画とモデル検査に基づく抜け穴検出を支えることを示す (6 節)。
- L_4 の表層構文の制御自然言語としての設計と、各選択の経験的動機を記録する (7 節)。
- 実装と、公的・私的部門における初期導入について報告する (8 節および 11 節)。

2 動機——LLM の時代に、なぜ言語か

2026 年に新たなプログラミング言語を提案すれば、もっともな反論が待っている。いまさら何のために、と。大規模言語モデル (LLM) はすでに重要なあらゆる言語でプログラムを書けるし、いわゆる「vibe coder」(雰囲気だけでコードを書く者) の一世代は、事実上もう手で書くのをやめ、意図を散文で述べてモデルに構文を埋めさせている。機械がコードを書くのなら、人間が読むための新しい記法をなぜ設計するのか。

答えは、法は普通のソフトウェアとは違う、それも法が**噛みついてくる**ときに違う、という点にある。たいていの場合、市民は自分を律する規則について合理的な無知のままでいることに満足し

ている。税の表や保険証券を理解する費用の期待値が便益の期待値を上回るので、署名して先へ進むのだ。だが、この計算は、利害が具体的かつ個人的なものになった瞬間に反転する——その人が収監されかねないとき、あるいは 600 ドルの請求を予期して封を切ったら 6,000 ドルだったとき。その瞬間、普通の人々は、膨大で、しかも完全に合理的な、細部への能力を発揮する。彼らは規則を**自分自身で**理解したいと望む。もはやブラックボックス——それが官僚であれ、チャットボットであれ、実際の権利を追跡しているかどうか定かでない自信ありげな言い換えを返す不透明なモデルであれ——に委ねることをよしとしない。もっともらしく聞こえる生成された答えこそ、まさに信用できないものである。なぜなら、争いはいまや敵対的であり、相手方は語を別様に読むあらゆる動機をもっているからだ。

そうした状況が要求するのは、**二重に**読める規則である。すなわち、自分の状況に照らして推論を確かめたい非技術者が読める規則であり、かつ、それを評価し、テストし、幻覚なしに説明できる計算機が読める規則である。この二重可読性の要請は新しいものではない。それは、今でいう**ローコード**の繰り返し現れる志である。すなわちまさに、経営者が読めるものとして業務規則を表現しようとした COBOL や、アナリストが——エンジニアだけでなく——データに問いを発せられるように宣言的な表層をもたせた SQL を生んだ志である。どちらも、ある種の規則は黒白の文字で書き残すことを人々が常に求めるほど重要だ、という賭けに出た。L4 はその賭けを引き継ぎ、一つの精緻化を加える。すなわち、その黒白は**形式言語**であるべきだ、と。自然言語による起草は、回避可能な、自ら招いた欠陥——語彙の曖昧性、並列節における係り受けの曖昧性、悪名高い句読点の不安定さ^{*1}——を抱えており、設計された記法はそれらをそもそも禁じることができる。本稿のアーキテクチャにおいて LLM が担うのは、この記法を置き換えることではなく、それを著し、説明するのを助けることである。すなわち、原文から最初の形式化を起草し、厳密な評価トレースを利用者にわかりやすい散文へと言語化する。神経的な「右脳」が言語を、決定論的な「左脳」が規則を担う。L4 は左脳である。

3 背景と設計目標

■**制定的規則と規制的規則** Searle は、先行して存在する行動を統制する規制的規則（「左側を運転せよ」）と、その行動の可能性そのものを創り出す制定的規則（「この駒の配置はチェックメイトとして数えられる」）とを区別する [50]。Hart の一次・二次規則や、Hohfeld の法的関係も近縁の線を引く [25, 30]。この区別は、形式化を行う者にとって学問上のものにとどまらない。制定的規則——「施行後に英国で生まれた者は、……ならば英国市民である」——は定義であり、事実を法的分類に関係づけるものであって、自然に状況から結論への**関数**となる。規制的規則——「買主は 30 日以内に支払わなければならない、これを怠れば……」——は、時間の経過にわたる行為、非難、そして救済に関わり、自然に**プロセス**となる。両者は異なる数学を要求する。単一形式体系に

^{*1} コンマの欠落をめぐる事件は一つのジャンルを成している。O'Connor v. Oakhurst Dairy は、連続コンマの不在が 500 万ドルの残業代紛争を左右した。

よる手法の常なる弱みは、二つのうち一方が他方に合わせて歪められることにある。義務論的概念が通常の述語として符号化されてその時相的・救済的構造を失うか、あるいは定義的計算が、それがふさわしくないモーダル論理に無理やり押し込められるのである。L4 の中心的なアーキテクチャ上の決断は、それぞれに固有の層を与えることだ。

■**同型性** 英国国籍法を論理プログラムとして表した伝統 [51] と、より広い rules-as-code 運動 [41] に従い、L4 は**同型** (isomorphic) な符号化を目指す。すなわち、形式テキストの構造は原文の構造を反映すべきである。ある条が (a)(b)(c) の段落を「かつ」「または」で結んでいるなら、符号化も同じ連言・選言の木を呈し、分野の専門家が条項ごとに対応を監査でき、後の改正がコードの局所的変更に対応するようにすべきである。同型性は正しさの規律であると同時に保守の戦略でもあり、また表層構文のいくつかの選択を導く (7 節)。

■**二つの読者** L4 は、相容れない習慣をもつ二人の読者を満たさねばならない。機械は、表示的意味論を備えた曖昧さのない文法を望む。弁護士は散文を、あるいは厳密さが許す限りそれに近いものを望み、記号には我慢がならない。本稿の主張は、これらの目標が対立ではなく両立可能だということである。言語は、完全に通常の形式的意味論をもちながら、**同時に**、制御自然言語として調律された表層構文をもちうる。4 節から 6 節が前者を、7 節が後者を論じる。

4 制定的中核——型付き関数型言語

L4 の制定的中核は、静的型付きで全域的な高階関数型言語である。型は DECLARE で導入する。IS ONE OF による列挙型・代数的データ型と、HAS によるレコードである。

```
DECLARE RiskCategory IS ONE OF
  LowRisk
  MediumRisk
  HighRisk
  Uninsurable
```

```
DECLARE Driver HAS
  name          IS A STRING
  age           IS A NUMBER
  accidentCount IS A NUMBER
  hasTickets    IS A BOOLEAN
```

関数は、入力の名指す GIVEN と結果の名指す GIVETH (同義語 GIVES) から成るシグネチャに続き、MEANS による定義を、あるいは真偽値をとる規則では DECIDE... IF を置いて導入する。パターン照合には、型安全で網羅性が検査される場合分けである CONSIDER / WHEN を用いる。

```
GIVEN driver IS A Driver
GIVETH A RiskCategory
`assess risk` driver MEANS
  CONSIDER driver's accidentCount
  WHEN 0 THEN IF driver's hasTickets
    THEN MediumRisk
```

```

ELSE LowRisk
WHEN 1 THEN MediumRisk
WHEN 2 THEN HighRisk
OTHERWISE Uninsurable

```

この中核の三つの性質が法にとって重要である。**全域性と網羅性**——型検査器は場合を尽くさない CONSIDER を拒絶するので、形式化された規則がある区分の申請者を黙って見落とすことはありえない。未処理のエッジケースに相当するものは、誤った給付の小切手ではなく、コンパイル時のエラーである。**型安全性**——支払額の計算式が日付を通貨に誤って加えることはできない。**説明可能性**——中核が純粋な関数型評価器であるため、あらゆる結果は、最上位の問いから決め手となる部分条件に至るまでのトレースを伴い、各ステップにはそれを正当化した規則の注釈が付く。このトレースこそ、システムが利用者に示すものであり、また、法そのものについてモデルに推論させるのではなく、地に足のついた事実として言語モデルに手渡すものである。

■同型符号化の例 AI・法分野の代表的ベンチマークである英国国籍法の一断片を考えよう [51]。法令の入れ子になった「または」と「かつ」が符号化に直接現れ、識別子は法令の句そのものである（バッククォートにより識別子に空白を含められる）。

```

GIVEN p IS A Person
GIVES A BOOLEAN
DECIDE `is a British citizen` IF
    `is born in the United Kingdom after commencement` p
    OR `is born in a qualifying territory after the appointed day` p
    AND
        `is a British citizen` OF `father of` p
        OR `is a British citizen` OF `mother of` p
    OR `is settled in the United Kingdom` OF `father of` p
    OR `is settled in the United Kingdom` OF `mother of` p

```

ここで、真偽値の木の優先順位を担うのは括弧ではなく字下げである点に注意したい（7 節で立ち返る）。@export を付した関数は、REST エンドポイント、OpenAPI スキーマ、そして LLM エージェントが呼び出せるツールへと自動的に持ち上げられる。したがって単一のソースファイルが、各分岐を決定した規則への引用付きで、対話的な適格性ウィザードを生み出す。

5 規制層——並行プロセスとしての契約

定義的関数では義務を表現できない。「買主は支払わねばならない」は、ある状況について真でも偽でもない。それは将来の行為に対する要求であり、事象が展開するにつれて履行されたり違反されたりし、場合によっては救済を引き起こす。このために L4 は、五つのキーワードからなる骨格を備えた規制的部分言語を提供する。

```

PARTY actor
MUST action -- または MAY, SHANT, DO
WITHIN deadline
HENCE successContinuation
LEST failureContinuation

```

```

DECLARE Person IS ONE OF Seller, Buyer
DECLARE Action IS ONE OF
  delivery
  payment HAS amount IS A NUMBER

saleContract MEANS
  PARTY Seller
  MUST delivery
  WITHIN 3
  HENCE ( PARTY Buyer
    MUST payment 100
    WITHIN 7
    HENCE FULFILLED
    LEST BREACH BY Buyer BECAUSE "payment not made within 7 days" )
  LEST BREACH BY Seller BECAUSE "delivery deadline exceeded"

#TRACE saleContract AT 0 WITH
  PARTY Seller DOES delivery AT 2
  PARTY Buyer DOES payment 100 AT 5
-- 結果: FULFILLED

```

図 1 規制的契約としての二者間売買。HENCE が売主の引渡しを買主の支払義務へと繋ぐ。
#TRACE は時刻付き事象列を再生し、FULFILLED、非難を帯びた BREACH、あるいは残存する義務を報告する。

MUST, MAY, SHANT は義務論的様相（義務・許可・禁止）であり、DO は拘束を伴わない行為である。WITHIN は期限を付す。HENCE と LEST は、義務がそれぞれ履行された場合・されなかった場合にとられる継続であり——決定的なことに、何が履行に数えられるかは様相に依存する。MUST では、期限までに行為をなせば HENCE の枝を、期限を徒過すれば LEST の枝をとる。SHANT では極性が反転し、禁止はその行為のないまま期限が過ぎたときにこそ守られる。LEST は典型的には救済を連鎖させる。それは違反を償う後続の義務であり、標準義務論理を悩ませる義務違反時（contrary-to-duty）構造の形式的な居場所である [9, 54]。

■**トレースに基づく意味論** 規制的契約の表示的意味は、時刻付き事象トレースから結果への関数である。結果とは FULFILLED か、非難と理由を担う BREACH か、あるいは何が依然として負われているかを記述する**残存契約**である。これはまさに、Hvitved が契約仕様言語（CSL）において多者間契約に与えるトレースに基づくモデル [32, 33] であり、それ自体、Andersen, Elsmann, Henglein, および Peyton Jones と Eber の合成的な商契約・金融契約 DSL の末裔である [3, 44]。契約とは、それに適合する事象トレースの集合である。適合性検査はトレースの所属判定であり、残存とは事象の接頭辞を消費した後の契約の状態、すなわち「いま誰が、何を、いつまでに、なお負っているか」への機械可読な答えである。L4 の#TRACE（図 1）は、まさに観測された事象を表示的意味へ積み込むこの操作である。我々は CSL から多くを直接借りている。義務・許可・禁止の目録、期限と救済（義務違反時）構造、そして何より、契約とはそれに適合するトレースの集合であるという

表 1 L4 の規制のキーワードとその CSL / CSP における対応物。

L4	CSL / CSP の対応物
PARTY p MUST a HENCE k	番兵付き前置 $p.a \rightarrow k$
WITHIN t	時間付き期限 (Timed CSP のクロック番兵)
HENCE k	成功継続
LEST k'	失敗・時限切れ継続; 救済 (義務違反時)
A RAND B	同期並行 $A \parallel B$
A ROR B	選択 $A \square B$
FULFILLED	成功終了 (SKIP)
BREACH	非難を割り当てる終了
#TRACE c WITH es	トレース所属判定 $es \in \text{traces}(c)$

根本的な立場である。CSL の契約 DSL に対して L4 が加えるのは、それが意図していなかった二つのもの——制御自然言語の表層と、統合された制定的関数型中核——であり、これにより契約が分岐の根拠とする述語自身が、不透明な原子ではなく、型検査された実行可能な法となる。時刻付き事象が義務を発生・消滅させる物語として契約を捉えるこの双対的な見方は、事象計算 [35] の設定そのものでもあり、我々はこれを同じトレースの論理的読みとして採る (6 節)。

■CSP の系譜 作用的に読めば、規制層は Hoare の通信逐次プロセス (CSP) の伝統に連なる小さなプロセス計算である [28, 29, 48]。対応は直接的である (表 1)。義務 PARTY p MUST a HENCE k は、番兵付きの事象前置 $p.a \rightarrow k$ である。HENCE と LEST は、環境——トレース——が実際に何をなすかによって解決される選択を成し、CSP の外部選択にあたる。RAND は完了において同期する並行合成であり、どちらか一方が違反すれば合成全体が違反する。ROR は部分契約間の選択である。FULFILLED は成功終了 (SKIP) であり、BREACH は非難を割り当てる終了である。唯一の本質的な追加は時間である。WITHIN は期限を課し、L4 を Timed CSP[47] や、UPPAAL のようなツールの基礎をなす時間オートマトン [2, 5] とともに位置づける。再帰は反復的な義務を与える。ローンの月々の分割払いは、HENCE の枝で次期の義務を、LEST の枝で違約金付きの義務を再発行し、FULFILLED を底とする関数である。

■なぜこれが重要か 規制層が時間付きの並行システムであるがゆえに、弁護士が契約について発する問いは、検証ツールが答えうる問いとなる。「ある条項が義務づける行為を別の条項が禁ずる状態は存在するか」という問いは、各当事者のプロセスの直積上の到達可能性の問い——競合状態、義務論的にいえば矛盾する抵触——である。我々の公的部門導入の一つ (11 節) では、まさにそのような義務論的・時相論理的な二重拘束が、現行の規制の中に自動的に発見された。ある稀な交錯のもとで、当事者が同一の行為について同時に義務づけられかつ禁じられていたのである。抜け穴探索を状態空間探索として捉え直すことが、次節で扱う関係的読みへの架け橋となる。

6 一つのプログラム，二つの読み——関数的と関係的

関数 $f : A \rightarrow B$ は，集合論的にはそのグラフ，すなわち関係 $\{(a, f a) : a \in A\}$ である。関数論理型言語はこの同一性を活用する。Mercury は述語に**モード**（どの引数が入力でどれが出力か）と**決定性**の注釈を付け，関係を要求されたモードに応じて効率的なコードへコンパイルする [52]。Curry は狭義化（narrowing）と残留（residuation）を通じて関数と関係を統合し，関数を未知の入力で走らせて望む出力を与える入力を系が探索できるようにする [4, 24]。L4 はこの見方を受け継ぐ。その制定的規則は，全域型付き関数であるがゆえに，前向きの評価器だけでなく関係的形式——ホーン節，あるいは規制層については遷移関係——にもコンパイルされる。すなわち，PROLEG 流のホーン節，あるいは規制層については事象とフルーメントの上の遷移関係であって，これは事象計算の流儀である [35]。Logical English [34] は，この宣言的基盤それ自体が自然言語の表層をまとうことを示した。L4 は逆方向から，型付き関数型プログラミングから出発してこの関係的読みを変換によって回復し，Mercury と Curry を二つのパラダイムを橋渡しする概念的な橋とする。

この第二の読みは余興ではない。それこそが，L4 を単なる実行のためのものではなく**分析**のための道具とする。三つの能力が従う。

後ろ向き・仮説形成的問い合わせ 前向きに走らせれば，``is a British citizen``は与えられた人物を分類する。後ろ向きに走らせれば，同じ規則は市民権が成り立つ状況の特徴づける。これは説明（「どの事実を変えればこの判断は反転するか」）にも，規制層における計画にも有用である。「ローンの申込みは，なお間に合うように行える**最も遅い**時点はいつか」という問いは，契約の遷移関係上の仮説形成的な問いであり，論理プログラミングが本来的に答えるが一方向の評価器には答えられない種類のものだ。

網羅的なシナリオ生成 規則を関係として扱えば，性質に基づくテストが状況の空間を枚挙し，それらすべてにわたって不変条件を検査できる。法的なコードに通常付随する，手で書かれた一握りの例ではなく。ある初期実験では，元来ごく少数の単体テストしか伴わない Python の規則ライブラリであった国家給付の計算を関数型の環境へ取り込み，シナリオの領域全体にわたるテストを生成して，手書きの疎なテスト群が見落としていた誤計算を表面化させた。

モデル検査 遷移関係は，記号的モデル検査器——NuSMV, SPIN, TLA+, UPPAAL [10, 31, 37, 5]——に手渡して，仕様レベルで書かれた性質に違反する状態を探索させることができる。ここでは，法の文言が対象レベルに，法が精神が性質の表明として存在する。反例トレースは抜け穴であり，セキュリティ研究者が脆弱性を見つけるのと同じ仕方で見つかる。5 節の義務論的・時相論理的抵触は，そうした反例の一つである。

L4 の実装はすでにこの多目的戦略の萌芽を備えている。問い合わせプランナは関係的ゴールを並べ替え，MLIR/WASM 基盤は判断論理を可搬なバイナリへコンパイルする（8 節）。原理的に重要なのは，分野の専門家が**一度著した単一**の同型ソースが，これらすべての分析を養うことである。関数的読みと関係的読みは，同一テキストの二つの眺めであって，同期を保つべき二つの成果

物ではない。

7 制御自然言語としての表層構文

ここまではすべて意味論に関わり、それを担う具象構文はいくらでもありうる。L4 の賭けは、具象構文こそが採用の成否を決する場であり、形式言語を、その形式的性格を犠牲にすることなく**制御自然言語**——制限されてはいるが厳密な英語の断片 [36]——へと整形できる、という点にある。設計はいくつかの原則に従い、各々に固有の動機がある。

■**記号ではなく語** L4 は、句読点や記号よりも馴染みのある語を好む。≥ には AT LEAST, + には PLUS, フィールド参照には 's (driver's age), 適用には OF. 型シグネチャは Person → Bool としではなく、英語の節——「人物を GIVEN とし、真偽値を GIVES」——として読める。代償は冗長さであり、便益は、型シグネチャを見たことのない読者でも文として解せることである。これはまさに 2 節の二重可読性の要請に資する。属格の接語 's は、この精神における小さな、しかし心地よい多重定義である。英語は 's を所有のためにも、また is (である) の短縮形としても書く。ゆえに driver's age は、運転者がもつフィールドという has-a として自然に読める一方、まさに同じ接語が is-a の述語付けにおいて繫辞として注釈されうる——オブジェクト指向が合成 (composition) と部分型付け (subtyping) として別々に保つ二つの関係を、一つの記号がちょうど跨ぐのである。

■**二つのレジスタ** 記号を置き換える語は大文字で組まれる。この選択は単なる装飾ではなく、表層を二つの視覚的レジスタに分ける。UPPERCASE のトークンは**言語**の固定語彙——AND, OR, NOT, AT LEAST, GIVEN, MEANS——であり、'バッククォートで囲った小文字の句'は**法**の語彙、すなわち原典から採られた用語である。ゆえに読者は、文法をまだ知らずとも、どの語が論理的な力を持ち、どの語が分野の概念を名指すだけなのかを一目で見分けられる——COBOL や SQL がキーワードをプログラマ自身の識別子と区別するのに用いたのと同じ組版上の慣行である。大文字・小文字の区別が、文書化の役割をも兼ねるのである。

■**括弧ではなく字下げ** オフサイド規則 [38] に従い、L4 はレイアウトに敏感であり、そのレイアウトを特定の法的用途に供する。連言・選言の字下げが真偽値の木の優先順位を符号化するので、コードの視覚的な形が規則の論理的な形となる。4 節の英国国籍法の断片では、AND の下と横に OR が揃えられていることが構造を曖昧さなく定める。読者は括弧の対応をとる必要がなく、字下げを原文の段落番号に反映させて同型性を進めることができる。これはプログラミングの習慣を法へ持ち込むというより、むしろその逆である。字下げされ列挙された号は、立法起草が整えられた一九世紀の改革以来、書かれた法が組まれてきた形そのものであり [12], ワードプロセッサも法令の箇条書きを同じように字下げする——L4 はその字下げに優先順位を担わせるだけである。

■**不活性な足場と無連結** 形式化が原文をほぼ一語一語再現できるようにする装置がさらに二つある。法令が各行に接続詞を繰り返さずに号を並べる場合——起草された箇条書きに典型的な**無連結** (asyndeton) の列挙——に対し、L4 は... (三つの点) と.. (二つの点) を、それぞれ AND と OR

```

`grant is allowed` MEANS
    "the grant is allowed where all of the following hold---"
    ... `the form was filed on time`
    ... `the fee was paid`
    AND `the applicant is eligible`

`is eligible` MEANS
    "the applicant is eligible if any of---"
    .. `a citizen`
    .. `a permanent resident`
    OR `the holder of a valid pass`

```

図2 無連結と不活性な足場。... は暗黙の AND, .. は暗黙の OR であり、最後の号は起草された箇条書きと同様に接続詞を明示する。引用符付きの柱書きは不活性で真偽値に寄与せず、原文をそのまま規則へ運ぶ。

```

myDeal MEANS
  Deal WITH
    buyer IS Party WITH name IS "Alice Avocado"
    seller IS Party WITH name IS "Robert Rich"
    item IS LIST InventoryItem OF "potato", 12, Nothing
      ^ OF "corn", 120, Nothing
      ^ OF "avocado", 2, Nothing
    price IS Money OF usd, MoneyAmount OF 100, 00

```

図3 ditto 用キャレット^は直上のトークンを繰り返し、揃えられた表組みのデータを列として読ませる。

の軽量な中置の同義語として提供する。点の数は意図的であり、AND の三文字には三つ、OR の二文字には二つを当て、等幅のレジスタにおいて暗黙の記号とその明示形が同じ桁を占め、各号が揃うようにしてある。こうして接続子は継続行の先頭に控えめに置ける。また、真偽値の位置に現れた引用符付きの文字列は**不活性** (inert) として扱われる。すなわち AND の内側では TRUE に、OR の内側では FALSE に評価され、いずれにせよ周囲の条件の真偽値を変えない。不活性なテキストにより、起草者は原文の柱書きや接続の文言——「著作者の生存中および」「いずれか早く満了するもの」——を、型検査器が受理し評価器が無視する素材として、そのまま規則に貼り込める。レイアウトと相まって、これらは列挙された条項を号ごとに書き写すことを可能にし、それはコードが法の述べることを述べていると弁護士が認めるための表層上の条件である (図2)。

■Ditto 記号 法的・財務的な表組みのテキストは構造を列方向に繰り返す。書記は古くからその繰り返しを ditto 記号で略記してきた。L4 はこの慣行をキャレット^で借りる。これは直上のトークンを表し、揃えられた表組みのデータ——領域の一覧、在庫、料金表——を、人が帳票を埋めるように、繰り返される素材を列へ畳んで書けるようにする (図3)。要点は打鍵の節約ではなく認知的負荷であり、目が列を一つの単位として読む。

```

GIVEN `the worker` IS A Person
      `the firm`   IS A STRING
GIVES A BOOLEAN
`the worker` `works for` `the firm` MEANS
      `the worker`'s employer EQUALS `the firm`

DECIDE `Alice is entitled to leave` IF
      `Alice` `works for` "Acme"

```

図 4 ミックスフィックス。キーワード `works for` が二つの引数の間に割り込むので、定義も呼び出し (`Alice` `works for` `Acme`) も `worksFor(...)` ではなく英語の節として読める。

■句としての識別子とミックスフィックス バッククォートで囲った識別子は空白を含みうる——バッククォートでリテラルなテキストの範囲を囲うという、Markdown の書き手には馴染みの慣行を借りている——ので、識別子が法令の句そのもの (`is settled in the United Kingdom`) になりうる。そうした名前はミックスフィックスでありうる。引数が名前と交互に並び、述語が文として読める——`worksFor(employee, employer)` ではなく `employee` `works for` `employer` である。これらが相まって、形式化が原文を引用できるようになる。これは、コードが法の述べることを述べていると弁護士が認めるための、表層上の条件である (図 4)。

これらの選択が代償なしだとは主張しない。冗長さ、レイアウト敏感性、句としての識別子は、それぞれ簡潔さやエディタ支援において何かを犠牲にする。だが我々は、それらが、最初の読者が分野の専門家である言語にとって正しい選択であること、そしてそれらが表層においてなされ、4 節から 6 節の意味論を手つかずに残すことを主張する。

■系譜 L4 の表層設計は、規則のための制御言語・計算言語の豊かな伝統を受け継いでいる。そのローコードの先祖は COBOL と SQL (2 節) であり、大文字で英語の形をした表層が非プログラマに業務規則を読ませうという賭けに最初に出た。以後の制御言語の伝統はその賭けを精緻化してきた。Grammatical Framework [46] からは、表層構文を型理論的文法——抽象構文とその具象的实现——として扱う規律を、すなわち同一規則の多言語的描画への道を採用。C&C と Boxer の広被覆構文解析の伝統 [14, 8] からは、逆方向に走らせて言語を論理形式へ写すという、いまや LLM が容易にする取り込みの目標を採用。企業標準である SBVR[42] と DMN[43] からは、プログラマだけでなく業務アナリストの間にも、構造化された語彙・規則・判断表への確かな需要があることを学ぶ。そして、業務規則の制御言語を SBVR へコンパイルする RuleCNL[20] のような CNL の試みからは、制御された表層が、形式的中核を露出させることなくそれを標的にできるといふ具体的な教訓を得る。これらに対する L4 の独自の賭けは、CNL の表層を、制定的関数と規制的契約の双方にまたがる実行可能で型検査された意味論と組み合わせ、かつ表層を、業務アナリストの語彙ではなく法文に対して同型にすることである。

8 実装とツール

L4 は Haskell で実装されている。中核 (jl4-core) は、レイアウトに敏感な構文解析器、Hindley–Milner 流の型検査器、そして 4 節の説明的トレースを生成する評価器から成る。その周囲に、単一の .14 ファイルを生産的にするためのツール群が配される。エディタ内での型検査・インライン評価・定義への移動のための言語サーバ (LSP)、トレースと問い合わせプラン用コマンドを備えた REPL、`@export` 注釈付き関数を REST エンドポイントおよび LLM エージェント向けの Model Context Protocol ツールとして公開する判断サービス、ブラウザ内・サーバレス評価のための WebAssembly ビルド、そして判断論理を可搬な .wasm バイナリへコンパイルする MLIR 基盤である。ツールチェーンはあらゆる評価を GraphViz のラダー図として描画し、規則集合を人間によるレビューのために Markdown へトランスパイルし、同一ソースから消費者向けのウェブ・ウィザードを生成できる。静的解析器は金融契約を ACTUS および FIBO 標準に照らして分類する。jl4.legalese.com のウェブエディタは、このパイプライン全体をクライアント側で走らせる。

このスタックにおける LLM の役割は意図的に限定されている。モデルは**取り込み**——PDF や法令から最初の L4 形式化を起草すること (9 節)——と**説明**——完了した決定論的な評価トレースを素人の利用者のために言語化すること——を支援する。モデルは推論のループの中にはいない。推論器は L4 の評価器であり、その各ステップは監査可能である。モデルの自然言語による出力は、そのトレースによってガードレールを課され、それに照らして検査されうる。これが、L4 が法的 AI における幻覚に対抗する仕方である。モデルに法について推論させることを信頼するのではなく、モデルに語るべき厳密な成果物を与えるのである。

9 取り込み——知識獲得ボトルネックの再考

1980 年代のエキスパートシステムは**知識獲得のボトルネック**に座礁した [19, 27]。ある領域を符号化するには、知識エンジニアが分野の専門家との緩慢で高価な対話を要し、しかも出来上がる脆いシステムが労力に見合うことはまれだった。法的エキスパートシステムも例外ではない。英国国籍法を論理プログラムとして表した仕事の見事さ [51] にもかかわらず、希少なプログラマと弁護士の間によって一度に一つの法令を形式化していくやり方は、法そのものの規模には決して近づかなかった。L4 はこのボトルネックに二方向から挑む。

■**手作業で、ただしより良く** 手作業の符号化でさえ、同型性を第一級の目標とする制御自然言語として作られた言語では容易になる (3 節・7 節)。形式テキストが原文の構造を反映しその文言を引用するので、分野の専門家はプログラマにならずとも条項ごとに監査できる。我々はこの架け橋が現実には荷重を支えることを見出した。非技術者の業務担当者が、L4 の規則集合を**読み**、それを自分たちが意図する規則の忠実な言明として**承認**できるのである。これは重大な成果である。なぜなら、承認とはまさに、コードを書いたエンジニアから、規則を所有する責任者へと法的責任が移

される行為だからだ——その分野の専門家にとって不透明な形式体系には決して築けない架け橋である。知識エンジニアと専門家の組が消えるわけではないが、その間の溝は狭まる。そして我々の経験が示唆するように、要件分析を仕込まれたソフトウェアエンジニア自身が、業務の専門家が読み飛ばすであろう曖昧さを表面化させる有能な言葉の職人である。強い型と網羅性検査は、欠けた場合を本番ではなく符号化の瞬間に捕らえる。手作業の符号化にできないのは規模化である。それは希少な専門家の時間に対して線形のままにとどまる。

■**規模において、型システムを神託として** 大規模言語モデルは最初の一手の経済を変える。モデルは法令や規程の形式化を数秒で提案でき、人間の役割を著述からレビューへと移す。危険はおなじみのもの——流暢だが微妙に誤った符号化——であり、ここで L4 の静的型システムと言語サーバが決定的なガードレールとなる。型検査器は、型の合わない・網羅的でない形式化を即座に拒絶する健全で決定論的な神託であり、LSP は各違反を正確な位置とともにインラインで報告する。そうしてモデルは、ファイルが型検査を通るまで、さらにその `#ASSERT` のゴールデンテストが通るまで、生成・検査・修復のループの中で駆動でき、神託は型の正しさから振る舞いへと拡張される。これは本稿の神経記号的 (neuro-symbolic) 主張を取り込みそれ自体に適用したものである。LLM が提案し、型システムが処理する。強い型付けは、開かれた自然言語からコードへの生成を検証器に対する制約付き探索へと変える。これは型主導のプログラム合成を扱いやすくする規律と同じであり、モデルの自信に正しさを明け渡すことなく取り込みを規模化させる。

■**規模における資料体** 型でガードされた取り込みのループにも、なお原材料——機械可読な一次法令——が要る。補完的な実現要因は、それを解放しようとする数十年来の運動である。Carl Malamud の [Public.Resource.Org](https://public.resource.org/) と [Law.gov](https://www.law.gov/), そして法とそれに組み込まれた標準が著作権の背後に閉ざされてはいないことを確立した訴訟 [39] は、膨大な一次法令資料を公に利用可能にしてくれた。その系譜に連なるプロジェクト——Legal Data Hunter の取り組みなど——は、いまや法令・規則・法典を規模において取り込み可能な形へと送り出している。一方に開かれた資料体を、他方に型でガードされた LLM のループを置けば、法の同型形式化は、かつて知識獲得ボトルネックが閉ざした規模において、扱いやすく見えてくる。とはいえ、それで自動になるわけではない。構造ではなく立法上・契約上の意図への忠実さは、依然として人間の判断と承認の問題である (11 節)。しかし、形式化はもはや、希少な組に律速されて一度に一つの法令ずつ進む必要はない。

10 応用

単一の .14 ソースが多く基盤を養う (8 節) がゆえに、一つの形式化は、単一の製品ではなく、法のライフサイクル全体——起草、分析、市民の自助、運用上の判断——にわたる応用の幅を支える。四つの原型を素描する。11 節の実証はそれらを具体化したものである。

■**市民向けエキスパートシステム** 型検査器が検証するのと同じソースから、ツールチェーンは対話的なウェブ・ウィザードを自動生成する。市民が自分の状況の事実を入力すると、自分の義務や

権利が、決め手となった規則への引用と、平易な言葉での説明としての評価トレースを伴って示される。これは 2 節の破壊的な楔である——署名はしたが読まなかった規制や証券のもとで、自分がどこに立っているかをただ知りたい、過剰に供給された非消費者である。L4 では、そのようなエキスパートシステムを構築することは、そもそも規則を書き下したことの副産物であって、別個のエンジニアリング事業ではない。

■**形式検証と静的解析** 関係的阅读（6 節）は、規則集合や契約の草案を、モデル検査基盤——NuSMV, SPIN, TLA+, UPPAAL——に手渡してコンパイル時の静的解析にかけることを可能にする。すなわち、競合状態や義務論的・時相論理的な二重拘束、決して発火しえない条項や決して履行しえない義務、そして起草中の立法や交渉中の契約における明白な矛盾の発見である。抜け穴探しが脆弱性探しになる。同じ機構は、各当事者の「最も気にかける十数のシナリオ」を、改訂のたびに走る回帰テスト群へと変える。これにより、いずれの側からの編集であれ、以前合意した性質を壊すものは、後から争われるのではなく直ちに捕らえられる。

■**自然言語へ戻す生成的起草** 取り込みの方向——自然言語から形式的規則へ——は LLM の自明な用途だが、逆方向こそより興味深い。規則集合が形式的になり、**無矛盾であると証明されたなら**、生成的 AI はそれを流暢な英語へ描き戻せる。議会が審議し制定できる法令の散文、あるいは相手方が署名する条項である。ここでは検証された形式的成果物が真実の源であり、英語はその忠実に再生成可能な描画である——rules-as-code の議題が長く求めてきた反転 [41] であって、散文が権威をもち論理が後付けの注釈として黙って食い違わない。モデルは固定された成果物を言語化するよう制約されるので、散文が論理から逸れることはなく、文法工学的な手法 [46] を介して、同一規則を一つのソースから複数の自然言語へ描画できる。

■**監査に耐える企業的意思決定** 反対の大量処理の端では、判断サービスが企業の業務規則エンジンに接続し、運用上の判断——ローン申込みの承認・否認、保険金請求の査定、給付の適格性判定——を担う。差別化要因は処理能力ではなく説明責任である。あらゆる判断が、最上位の問いから決め手となる条件に至るまでの決定論的なトレースを伴うので、結果は**構成上説明可能かつ監査可能**である。これは、自動意思決定に関する GDPR の規定が示唆する [17]——その正確な射程は争われているが [55]——透明性の期待に、そして EU AI 法が高リスクシステムについて具体化するもの [18] に、直接応える。申請者は理由を受ける権利があり、L4 は理由を第一級の成果物として産出する。それは、事実上その人について「勘で」判断したモデルの事後的な合理化ではなく、引用可能な導出である。

これらの原型は四つの製品ではなく、一つのソースの四つの眺めである。プラットフォームとしての野心（13 節）はまさに、一度著された規則の上に、エコシステムがこれらすべてを、そしてまだ想像されていない他のものを構築できるということにある。

11 経験と未解決の問題

L4 は公的・私的部門にまたがる実証で試されてきた。ある政府機関とともに、規制遵守のために副次的立法の一片を符号化し、市民が自分の状況を入力して義務を見られるウェブ・ウィザードを生成した。各回答は出典規則への引用を伴い、形式検証の工程が 5 節で述べた義務論的・時相論理的抵触を露わにした。私的部門では、ある大手保険会社の証券の形式化が、相当の財務的エクスポージャを伴う支払額計算式の曖昧性を明らかにした。ある立法起草部局は、OECD が促してきた種の rules-as-code の実践として、将来の起草に L4 を検討してきた [41]。ある決済フィンテックは、料金表で密な商契約を L4 に取り込み、運用システムとの統合のために SQL 様の API を介して提供した。

これらの取り組みは、言語の粗削りな部分も露わにした。同型性のために重んじられるレイアウト敏感性は、空白を損なうワープロから貼り付ける著者を苛立たせることがある。句としての識別子は、トークン状の名前を前提とするエディタのツールに負荷をかける。規制層の、省略された HENCE・LEST に対する既定値は、便利ではあるが、「成功」の様相依存的な極性を内面化していない著者を驚かせうる。各々が生きた設計上の問いである。より根本的には、同型な形式化と忠実なそれとの隔たり——テキストの構造に合わせることと、その背後の意図に合わせることの隔たり——は、言語が単独で埋められるものではない。それは、本来そうあるべきように、人間の法的判断の問題にとどまり、いまやより良く道具で支えられている。

12 関連研究

L4 はいくつかの研究の流れの合流点に位置する。その貢献は、個々の流れというより、それらの組合せにある。

■**契約 DSL とプロセス計算** Peyton Jones と Eber の金融契約のパール [44] と、Andersen らの商契約の代数 [3] は、Hvitved の CSL[32, 33] がトレース意味論と非難の割当てを伴う多者間契約へと一般化した、合成的なコンビネータ流を確立した。L4 の規制層はその直系の末裔であり、制御自然言語の表層と制定的関数型中核との統合を加える。Stipula[13] もまた、状態と時間を伴うプロセス計算として法的契約をモデル化し、オートマトンの角度から契約＝プロセスの見方を補強する。

■**法の論理プログラミング** 英国国籍法プロジェクト [51] は、同型な立法符号化の基礎的な実演であり、PROLEG [49] と、より広い論証の伝統 [45, 6] がそれを継ぐ。Logical English[34] は L4 の制御自然言語の志を共有しつつ論理プログラミングの内にとどまる。L4 は、型付き関数型プログラミングから出発し、関係的読みを基盤としてではなく変換によって達する（6 節）点で異なる。

■義務論的・反証可能な推論 反証可能義務論理と Regorous の一連の仕事 [22, 23] は、規範的位置・許可・義務違反時構造を細心に扱う。L4 の LEST による救済は、同じ義務違反時現象 [9] を、実行可能性とトレース意味論への適合のために選んだ、意図的に作用的で表現力の劣る扱いである。

■法と契約のための言語 Catala[40] は精神において最も近い同時代の隣人である。立法のための言語であり、デフォルト論理に根ざし、強固な正しさの物語をもつ。L4 は、制定的・規制の明示的分割、並行性を意識した契約意味論、そして CNL の表層において異なる。DAML[15] は、Haskell 由来の関数型言語を台帳契約にもたすが、既存の法の同型形式化ではなく、分散台帳上の実行を標的とする。記号の起草の伝統は Allen[1] に、さらに遡れば立法の文を解剖した Coode の 19 世紀の仕事 [12] に連なる。

13 結論と今後の課題

本稿では、計算法学のための関数型プログラミング言語 L4 を提示した。L4 は、全域型付き関数として扱う制定的規則を、CSL と CSP の系譜に連なるトレースに基づく表示的意味をもつ時間付き並行プロセスとして扱う規制規則から分離する。我々は、単一の同型ソースが、評価と説明のための関数的読みと、仮説形成的計画とモデル検査に基づく抜け穴検出のための関係的読みの双方を許すことを示し、表層構文を制御自然言語として——句読点が少なく、レイアウトで構造化され、句で名付けられた——設計でき、同一テキストが弁護士にも機械にも読めることを論じた。動機は、それ自体のための技術的新規性ではなく、生成的 AI の時代に鋭くなった、まさに法が噛みついてくるときに人が自分自身で確かめられる規則への根強い需要である。

今後の課題は三つの軸に沿って走る。規制の意味論の十全な妥当性証明に向けた CSL および Timed CSP との形式的対応の深化、抵触検出と仮説形成的計画が非専門家にとってボタン一つになるような関係的基盤の成熟、そして、L4 の CNL 設計が法的に訓練された非プログラマの著者にとって障壁を実際に下げるかを測る統制された著作可能性の研究である。より広い目標はプラットフォームである。PostScript の流儀でスタックの底に言語を置き、その上に、一度、黒白の文字で書かれ、誰もが読める規則の上に、法的アプリケーション——ウィザード、検査器、計画器、統合——のエコシステムを構築できるようにすることだ。

謝辞

著者らは、計算法センターの同僚、および本稿で述べた実証に協力した法務・工学の協働者に感謝する。

付録 A L4 の PROLEG への拡張——立証責任モナド

本付録では、L4 を *PROLEG*[49]——佐藤健 (Sato) による、日本民法の「主要事実推定理論 (presupposed ultimate fact theory)」の論理プログラミング的描出——へと拡張する試みと、両者の間に我々が構築しつつある橋の理論を素描する。^{*2} PROLEG が標的として魅力的なのは、本稿の主題と整合する三つの理由による。それは形式化された法の大規模で実在の資料体（およそ 2,500 の規則からなり、数十年分の司法試験に照らして検証されている）であり、したがって規模における取り込みを試す (9 節)。それは純粋に制定的であり、したがって L4 の関数型中核とその関係的読みを行使する (6 節)。そしてその立証責任の固有の扱いは、本稿の本体が暗黙のままにしてきた区別——制定的定義でも規制的義務でもない第三の法的役割——を迫る。

A.1 正準 PROLEG

正準 PROLEG は、規則の矢印 $\leq$ と反証可能性演算子 `exception(H,E)` で拡張した通常の Prolog である。規則 H は、その反証子 E が証明可能なとき覆され、例外は入れ子になる。その上に、訴訟の語彙——`allege`, `provide_evidence`, `admission`, `plausible`——が重ねられ、各主要事実を誰がどの基準で立証せねばならないかを符号化する。図 5 は、正準的な賃貸借事例の核を示す。賃貸人の解除請求、転貸の承諾という抗弁、そして賃借人の非背信抗弁を覆す背信行為の例外——例外の例外——である。この事例は現実の日本法に由来する。民法 612 条は、賃借人が賃貸人の承諾を得なければ賃借権を譲り渡し又は賃借物を転貸できないこと、そして賃借人が無断で第三者に賃借物を使用・収益させたときは賃貸人が契約を解除できることを定める。最高裁昭和 41 年 1 月 27 日判決（民集 20 巻 1 号 136 頁）はこれに背信行為論（信頼関係破壊の法理）を付し、転貸が当事者間の信頼関係を破壊するに至らない特段の事情があるときは解除が認められないとした。これが事例の符号化する例外の例外である。条文の原文を次に掲げる。

民法第 612 条

賃借人は、賃貸人の承諾を得なければ、その賃借権を譲り渡し、又は賃借物を転貸することができない。

2 賃借人が前項の規定に違反して第三者に賃借物の使用又は収益をさせたときは、賃貸人は、契約の解除をすることができる。

A.2 二つの翻訳モード

我々は二つのモードで翻訳するが、重要なことに、それらは二つのコンパイラではない。

Mode B (判断のみ) 立証責任の層を消去し、各規則を純粋に制定的に読む。 $H \leq B$ は `DECIDE`

^{*2} 双方向の PROLEG $\leftrightarrow$ L4 構文解析器およびトランスパイラは、並行する取り組みにおいて活発に開発中である。以下は設計・理論のノートであって、完成したシステムに関する報告ではない。フロントエンドはすでに正準 PROLEG を以下で論じる事例上で往復させ (`parse . print = id`)、A.5 節の立証責任ライブラリは型検査を通り参照判決を再現する。Mode B のエミッタは作成中である。

```

cancellation_due_to_sublease <=
    agreement_of_lease_contract, handover_to_lessee,
    agreement_of_sublease_contract, handover_to_sublessee,
    using_leased_thing, manifestation_cancellation.
exception(cancellation_due_to_sublease, get_approval_of_sublease).
exception(cancellation_due_to_sublease, nonabuse_of_confidence).
nonabuse_of_confidence <= fact_of_nonabuse_of_confidence.
exception(nonabuse_of_confidence, abuse_of_confidence).
abuse_of_confidence <= fact_of_abuse_of_confidence.

```

図5 正準 PROLEG。賃貸借解除の規則集合 [49]。二つの反証子と例外の例外を伴う。

```

DECIDE `cancellation due to sublease` IF
    `agreement of lease contract`
    AND `handover to lessee`
    AND `agreement of sublease contract`
    AND `handover to sublessee`
    AND `using leased thing`
    AND `manifestation cancellation`
    AND NOT `get approval of sublease` -- exception
    AND NOT `nonabuse of confidence` -- exception

DECIDE `nonabuse of confidence` IF
    `fact of nonabuse of confidence`
    AND NOT `abuse of confidence` -- 例外の例外

```

図6 Mode B の L4。反証可能性は AND NOT となり、例外の例外は構造的に立ち現れる。出力は素の制定的 L4 であり、jl4-cli で検証される。

H IF B となり、一つの頭部に対する複数の規則は選言となり、各 `exception(H,E)` は連言される AND NOT E となる。図6は図5の Mode B 像である。この翻訳は層化された否定失敗 (negation-as-failure) の古典的な読み [11, 21] であり、符号付き依存グラフが層化されているとき、そのときに限り健全である——トランスパイラが仮定するのではなく検査せねばならない条件である。

Mode A (判決に忠実) 立証責任を保持し、それを L4 のライブラリとして具象化する。鍵となる観察は、Mode A が Mode B に、各事実を責任を担う値として型付けし `resolve` の工程を加えたものであるということだ。`resolve`こそが、まさに Mode A から Mode B への脱糖である。一つのコンパイラ、二つの忠実度である。

A.3 条項ごとの逐次解説

翻訳が同型の語に値するのは、読者が二つのテキストを並べ、条項ごとに照合できるときに限る。対応を明示するため、図5の賃貸借事例を、規則集合から事実基底、そして判決まで辿る。

```

admission(agreement_of_lease_contract, defendant). % +5 constitutive
allege(approval_of_sublease, defendant).
provide_evidence(approval_of_sublease, defendant). % but NOT plausible
allege(fact_of_nonabuse_of_confidence, defendant).
plausible(fact_of_nonabuse_of_confidence). % => established
allege(fact_of_abuse_of_confidence, plaintiff).
plausible(fact_of_abuse_of_confidence). % => established

```

図7 賃貸借の事実基底（抜粋）。相手方による `admission` と `plausible` はともに事実を立証する。`allege` されながら `plausible` とされない事実は未立証のままであり、その負担者がそれについて敗れる。

■規則 対応は局所的かつ構造的である。頭部を共有する二つの PROLEG 規則——`contract_end <= cancellation_due_to_sublease` と `contract_end <= expiration_...`——は、本体がそれらの選言である一つの DECIDE となる。規則と、その反証子を名指す exception/2 節——PROLEG はこれを別個のトップレベル節として書く——は、規則の前提に反証子ごとの AND NOT を一つずつ連言した本体をもつ単一の DECIDE へと集約される。この頭部ごとの集約が、翻訳の行う唯一の正規化であり、L4 の規則を、その頭部がいつ成り立つかの自己完結的な言明として読めるようにするものである。かくして六前提の `cancellation_due_to_sublease` 規則とその二つの例外は、図6の単一の宣言へ畳み込まれ、`nonabuse <= fact_of_nonabuse / exception(nonabuse, abuse) / abuse <= fact_of_abuse` の連鎖は、それを締めくくる二つの宣言となる。規則ごとに、L4 のファイルは PROLEG のそれと同じ形をもつ。

■事実 事実基底（図7）こそ、責任の語彙が働く場である。事実は、その主張者が基準を満たすとき——`plausible(F)` が成り立つとき——あるいは相手方が `admission` によって認めるとき、**立証済み**と数えられる。この規則を通して賃貸借の事実基底を読む。解除請求の六つの制定的事実は被告に `admission` されており、立証済みである。二つの承諾の事実は被告に `allege` され証拠も出されているが、決して `plausible` とされず、**未立証**である。一方、`fact_of_nonabuse` と `fact_of_abuse` はそれぞれ `plausible` であり、立証済みである。Mode B では、これらは DECIDE が消費する真偽値へ解決され、Mode A では、図11の `established / unestablished` の注入であって、各々がその負担者を台帳へ運ぶ。

■判決 `contract_end` の評価は、いまや規則を決定論的に降りていく。

- `contract_end` は `cancellation OR expiration` に簡約される。`expiration` の枝は立証済みの事実をもたず失敗するので、すべては解除にかかる。
- `cancellation` は、六つの制定的事実（すべて自白により立証済み）と、AND NOT `get_approval`, AND NOT `nonabuse` を要する。
- `get_approval` は二つの承諾の事実を要するが、これらは未立証なので失敗する——よって NOT `get_approval` が成り立つ。被告の第一の抗弁は退けられる。

- `nonabuse` は `fact_of_nonabuse` (立証済み) と `AND NOT abuse` を要する。だが `abuse` は `fact_of_abuse` を要し、これは**立証済み**なので、`abuse` が成り立ち、`NOT abuse` が失敗し、`nonabuse` が失敗する——よって `NOT nonabuse` が成り立つ。これが例外の例外である。原告の背信行為が被告の非背信の抗弁を覆すのだ。
- 二つの反証子がともに退けられ、`cancellation` が成り立ち、`contract_end` が成り立ち、賃貸人が勝つ。

この降下こそが L4 の評価トレース——8 節のラダー図——であり、図 11 の `#ASSERT` が検査するものそのものである。各ステップが原典に一对一で存在する規則を引くがゆえに、トレースは PROLEG プログラムの監査を兼ねる。レビューが読む説明は、法令それ自身の構造の上の文字どおりの歩みである。責任台帳はそこに、Mode B が捨てる次元を加える——`fact_of_abuse` は原告に、`fact_of_nonabuse` は被告に——各事実が立証されなかったなら誰が敗れたかという問いを、無償で回復しながら。

A.4 一階の規則と型再構成

賃貸借事例は命題的だが、PROLEG の大半は一階である。述語は位置で定まる型のない引数を担う。図 8 は第二の事例——未成年者による売買の取消しが強迫により覆される——の断片を示す。引数は買主・売主・契約・日付である。関数が型付きでフィールドが名付けられた L4 へこれを翻訳するには、**型再構成**が要る。`rescission_by_minor_buyer` の第一引数が、`minor` や `manifestation` へ流れ込む `Buyer` と同じ種であることを利用法から推論し、その位置に名前と型を与えるのである。これは述語シグネチャをまたいで走る Hindley–Milner 流の推論であり、型のない PROLEG と型付き L4 が最も目に見えて分かれる所だ。たとえば時間に関する引数は、不透明な原子ではなく、本物の順序をもつ L4 の `Date` 値となる (5 節)。

再構成は単なる便宜ではない。それは**リント**である。まさにこの例のために公刊された資料には誤記がある——反証子が `minifestation_by_duress` と綴られ、変数が `Maniester`・`Mnifestee` となっている——。型のない Prolog はこれらを黙って受け入れる。綴り違いの関手は、単に決して単一化しない新規の述語 (決して発火しえない反証子、それゆえ判決を静かに変えるバグ) だからだ。L4 の名前解決器と型検査器は、まさにこれらを未束縛または型不一致として拒絶する。9 節の型を神託とする取り込みのループは、ここで第二の配当を払う。PROLEG の資料体を L4 で往復させることが、資料体に自らを監査させ、大量の型のない規則が不可避に堆積させる静かな起草の誤りを表面化させるのである。

A.5 立証責任モナド

本稿が触れる三つの伝統——義務論的義務、残余的支配、立証責任——すべてに繰り返し現れる手は、規範的位置を責任ある当事者で添字付けること、(位置, 当事者) である。Hohfeld によれば [30], これらは三つの**相異なる**位置である。違反時に非難される義務負担者 (`duty`. CSL と L4 の

```

rescission_by_minor_buyer(Buyer,Seller,CID,Tc,Tr) <=
  minor(Buyer),
  manifestation(rescission(CID),Buyer,Seller,Tr),
  before_the_day(Tc,Tr).
exception(manifestation(A,Mfr,Mfe,Ta),
  manifestation_by_duress(Thr,Mfr,Mfe,A,Ta,Td,Tr)).

```

L4 (Mode B, 型を再構成) :

```

GIVEN buyer IS A Person, seller IS A Person,
  contract IS A ContractId,
  tContract IS A Date, tRescission IS A Date
GIVETH A BOOLEAN
DECIDE `rescission by minor buyer`
  buyer seller contract tContract tRescission IF
  `minor` buyer
  AND `manifestation` (rescission contract) buyer seller tRescission
  AND tContract `before` tRescission

DECIDE `manifestation` action mfr mfe tAction IF
  `manifestation fact` action mfr mfe tAction
  AND NOT `manifestation by duress` ... -- 強迫がこれを覆す

```

図 8 一階の PROLEG とその Mode B 像。位置で定まる型のない引数は、名前付き・型付きの仮引数へ再構成される。例外の例外（強迫が未成年者の取消しを覆し、売買が立つ）は翻訳を生き延びる。

PARTY ...MUST の主体的当事者、5 節）。隙間において決定する残余的支配の保持者（power）。そして、事実が立証されなければ**敗れる**非説得の危険の負担者（liability）である。ここで形式化するのは第三のものの**み**であり、トランスパイラを律する規則は、PROLEG の当事者が立証責任の役割であって L4 の義務へ**決して**合成してはならない、というものである。

ある命題の証拠的状態には独立な二次元がある。その**真理**（立証済み・反証済み・未決——自然に Maybe で、Nothing が未決の場合）と、その**責任**（誰が立証せねばならないか）である。「証拠的主体を担う値」は余読み手（coreader）・環境コモナド (*Subject, a*) [53] であり、それは主体がモノイドを担うとき、すなわち二つの責任がどう結合するかを決めたときに、ちょうど**モナド**となる。*plaintiff*◇*defendant* に意味あるものはないので、我々は主体を併合せず**蓄積**する。基本的な責任の上の自由モノイドは**義務台帳**であり、モノイド的出力を伴うコモナドはそのとき Writer モナド [56] となる。真理の次元の上に、証明に失敗してもなお台帳を保つ順序で——非難が失敗を生き延びるように——**積む**と、これは MaybeT (Writer [Obligation]) である（図 9）。Writer の出力として放たれる台帳こそが責任地図であり、合成によって無償で生成される。そして連言が短絡せず蓄積的であるがゆえに、その地図は構造的である（実行ではなく規則を反映する）。

```

data Subject      = Plaintiff | Defendant
data Obligation = Obligation { onWhom :: Subject, what :: Text }

-- truth (Maybe) over a ledger of burdens (Writer); MaybeT is
-- outside, so Provable a = ([Obligation], Maybe a)
type Provable = MaybeT (Writer [Obligation])

prove      :: Subject -> Text -> Maybe a -> Provable a -- a borne fact
notProven  :: Provable a -> Provable ()  -- negation as failure
flipBurden :: Provable a -> Provable a  -- involution = opposite(P)
absent     :: Provable a -> Provable ()  -- defeater, opposing party
resolve    :: Provable a -> Bool         -- close the world

```

図 9 立証責任モナド L4.Proleg.Burden。flipBurden は PROLEG の opposite(P) を例外境界へ局所化したものであり、absent = notProven。flipBurden は相手方が立証せねばならない反証子である。

A.6 L4 自身における構成

この構成は Haskell の理論にとどまらない。それは L4 自身で実現されている。図 10 は burden.14 からの抜粋である。L4 には Monad 型クラスがないので、モナドは評価器の中に宿り、同じ代数がソースレベルでは、真理の次元 (MAYBE BOOLEAN, NOTHING が未決の場合) と蓄積される義務台帳を担うレコードとして、明示的なコンビネータとともに立ち現れる。notProven は否定失敗であり、flipBurden は部分導出の義務を相手方へ付け替え、resolve は世界を閉じて NOTHING を FALSE にする——負担者が敗れる。属格のアクセサ p's truth——まさに 7 節の 's の多重定義——がレコードのフィールドを読む。

紙の上ではなく L4 で書くことの要点は、処理された判決が実行可能になることである。図 11 は図 5 の貸借借事例を直接符号化し、期待される結果を j14-cli が検査する #ASSERT として言明する。contractEnd と cancellation は成り立ち、承諾の抗弁と非背信の抗弁は成り立たない——後者は原告の背信行為によって覆される、例外の例外である。末尾の #EVAL は責任台帳を印字する。flipBurden がそれを双対化し、各事実をその裏を負う当事者へ付け替える。

A.7 推定はモノイドの単位元である

構造を半群からモノイドへ昇格させることは、半群が決して迫られない選択を迫る。すなわち単位元——いかなる証拠も結合される前の命題の値——である。法的には、この単位こそが推定であり、それを定めることは憲法的行為である。台 Bool は二つのモノイドを許す。互いに De Morgan 双対である (表 2)。(V, False) のもとでは、十分な根拠が真へ反転させるまで嫌疑は立証されない——無罪推定である。(∧, True) のもとでは、反証まで主張が立つ。同じ要素, 同じ演算, 異なる単位——異なる道德。flipBurden は一方を他方へ De Morgan 双対化し, $\neg\neg = \text{id}$ を映す。反証可能な推

```

DECLARE Subject IS ONE OF Plaintiff, Defendant

DECLARE Obligation HAS
  onWhom IS A Subject
  what   IS A STRING

-- Haskell: Provable = MaybeT (Writer [Obligation]). L4 has no Monad
-- typeclass, so the algebra surfaces as a record: a truth dimension
-- (NOTHING = undecided) plus an accumulating obligation ledger.
DECLARE Provable HAS
  truth      IS A MAYBE BOOLEAN
  obligations IS A LIST OF Obligation

GIVEN p IS A Subject, claim IS A STRING, mx IS A MAYBE BOOLEAN
GIVETH A Provable
prove p claim mx MEANS
  Provable WITH
    truth      IS mx
    obligations IS LIST (Obligation WITH onWhom IS p, what IS claim)

-- negation as failure: holds iff its argument is not established
GIVEN p IS A Provable
GIVETH A Provable
notProven p MEANS
  Provable WITH
    truth      IS (IF isTrue (p's truth) THEN NOTHING ELSE JUST TRUE)
    obligations IS p's obligations

-- relabel obligations to the opposite party (an involution)
GIVEN p IS A Provable
GIVETH A Provable
flipBurden p MEANS
  Provable WITH
    truth      IS p's truth
    obligations IS flipAll (p's obligations)

-- close the world: established => TRUE, else the burden default
resolve p MEANS isTrue (p's truth)

```

図 10 `burden.l4` の抜粋。L4 で書かれた立証責任の構成。レコード `Provable` は `MaybeT (Writer [Obligation])` の L4 像であり、コンビネータはソースレベルのモナド演算である。

定は単位元（証拠がそこから動かさうる）であり、**確定的推定**や**強行規範** (*jus cogens*) は吸収元（いかなる証拠も動かさない）である。したがって法的体制は対（単位元, 閾値）——誰が疑わしきの利益を受けるか、どれだけの疑いが許容されるか——である。刑事法は（無罪, 合理的な疑いを超えて）を定め、Blackstone の比率 [7] を、誤りの非対称な費用に対する事実上の決定理論的損失関数として符号化する [26]。モナドの単位律は法学的に読める。推定それ自体は何も動かさない。動かすのは証拠 ($\gg$) のみである。

```

-- defendant's first defence: alleged but NOT plausible => unestablished
getApproval MEANS
  conj (LIST (unestablished Defendant "approval_of_sublease"),
        (unestablished Defendant "approval_before_cancellation"))

-- plaintiff's exception-of-exception
abuse MEANS conj (LIST (established Plaintiff "fact_of_abuse_of_confidence"))

-- defendant's second defence, itself defeated by the plaintiff's abuse
nonabuse MEANS
  conj (LIST (established Defendant "fact_of_nonabuse_of_confidence"),
        (notProven abuse))

cancellation MEANS
  conj (LIST (established Plaintiff "agreement_of_lease_contract"),
        -- ... the other five constitutive facts ...
        (established Plaintiff "manifestation_cancellation"),
        (notProven getApproval),
        (notProven nonabuse))

contractEnd MEANS disj (LIST cancellation)

#ASSERT      resolve contractEnd
#ASSERT      resolve cancellation
#ASSERT NOT (resolve getApproval)
#ASSERT NOT (resolve nonabuse)

#EVAL contractEnd's obligations      -- the responsibility ledger
#EVAL (flipBurden abuse)'s obligations -- burdens flipped to the opponent

```

図 11 [49] の賃貸借判決を実行可能な L4 として。#ASSERT が報告された結果を符号化し、#EVAL が Writer 次元の蓄積する責任台帳を印字する。

表 2 命題ごとのモノイド単位元の選択としての立証責任。

Bool 上のモノイド	単位元	法的解釈
$(\vee, \text{False})$	<i>False</i>	無罪推定
$(\wedge, \text{True})$	<i>True</i>	主張の反証可能な推定

A.8 対応と解集合像

同じ非単調な内容は、橋が整合を保たねばならない三つの顔をもつ（表 3）。6 節のように L4 を関係化すること——A 正規形の平坦化によって関数を述語へ、出力引数を加えて——は、Provable とその否定失敗を解集合のデフォルト否定へ送る [21, 16]。安定モデルの枚挙はシナリオ探索となり、5 節の義務論的・時相論理的な二重拘束は充足不能核として再び現れる。これは、いまや立証責任の層を覆うまでに拡張された、同じ「抜け穴＝脆弱性」の枠組みである。賃貸借の事実基底を

表 3 同じ反証可能な内容の三つの顔。

	PROLEG	Provable	ASP
反証子	exception	notProven	not e
閉世界	暗黙	resolve	デフォルト否定
証明基準	plausible	Maybe 部分	弱制約
責任負担者	opposite	flipBurden	タイプブレーク
責任地図	—	Writer	当事者ごとのリテラル

符号化し、これらのコンビネータで `contract_end` を評価すると（図 11）、報告された判決が再現される。貸貸人が勝ち、転貸承諾の抗弁は十分な蓋然性を欠いて失敗し、非背信の抗弁は背信行為によって自ら覆され、台帳は各事実をその負担者に帰する——付随的に、**誰が責任を負うか**（制定的事実については原告）が、**誰がそれを立証するか**（ここでは被告の自白による）と独立であることを確立しながら。

本稿の主張にとっての含意は、3 節の制定的・規制的分割が第三の車線——証拠性——を得る、ということである。自然な利用者向けの成果物は、三つの射影（誰が責を負うか、誰が決めるか、誰が立証せねばならないか）をもつ**責任地図**であって、各々が異なる Hohfeld 的領域から引かれた（位置, 当事者）の対である。それらを別々の型としてもつことの価値は、まさに、トランスパイラが一方を他方へ黙って崩壊させられないことにある。

参考文献

- [1] Layman E. Allen. Symbolic logic: A razor-edged tool for drafting and interpreting legal documents. *The Yale Law Journal*, 66(6):833–879, 1957.
- [2] Rajeev Alur and David L. Dill. A theory of timed automata. *Theoretical Computer Science*, 126(2):183–235, 1994.
- [3] Jesper Andersen, Martin Elsmann, Fritz Henglein, and Ken Friis Larsen. Compositional specification of commercial contracts. *International Journal on Software Tools for Technology Transfer (STTT)*, 8(6):485–516, 2006.
- [4] Sergio Antoy and Michael Hanus. Functional logic programming. *Communications of the ACM*, 53(4):74–85, 2010.
- [5] Gerd Behrmann, Alexandre David, and Kim G. Larsen. A tutorial on UPPAAL. In *Formal Methods for the Design of Real-Time Systems (SFM-RT)*, volume 3185 of *LNCS*, pages 200–236. Springer, 2004.
- [6] Trevor Bench-Capon, Michał Araszkiewicz, Kevin Ashley, et al. A history of ai and law in 50 papers: 25 years of the international conference on ai and law. *Artificial Intelligence and Law*, 20(3):215–319, 2012.
- [7] William Blackstone. *Commentaries on the Laws of England, Book IV: Of Public Wrongs*.

- Clarendon Press, Oxford, 1769.
- [8] Johan Bos. Wide-coverage semantic analysis with Boxer. In *Proceedings of the 2008 Conference on Semantics in Text Processing (STEP)*, pages 277–286. College Publications, 2008.
 - [9] Roderick M. Chisholm. Contrary-to-duty imperatives and deontic logic. *Analysis*, 24(2):33–36, 1963.
 - [10] Alessandro Cimatti, Edmund Clarke, Enrico Giunchiglia, Fausto Giunchiglia, Marco Pistore, Marco Roveri, Roberto Sebastiani, and Armando Tacchella. NuSMV 2: An open-source tool for symbolic model checking. In *Computer Aided Verification (CAV)*, volume 2404 of *LNCS*, pages 359–364. Springer, 2002.
 - [11] Keith L. Clark. Negation as failure. In Hervé Gallaire and Jack Minker, editors, *Logic and Data Bases*, pages 293–322. Plenum Press, 1978.
 - [12] George Coode. On legislative expression; or, the language of the written law. In *Reprinted in E. A. Driedger, The Composition of Legislation*. Originally London: W. Benning, 1843.
 - [13] Silvia Crafa, Cosimo Laneve, Giovanni Sartor, and Adele Veschetti. Pacta sunt servanda: Legal contracts in Stipula. *Science of Computer Programming*, 225, 2023.
 - [14] James R. Curran, Stephen Clark, and Johan Bos. Linguistically motivated large-scale NLP with C&C and Boxer. In *Proceedings of the 45th Annual Meeting of the ACL, Demo Session*, pages 33–36. Association for Computational Linguistics, 2007.
 - [15] Digital Asset. DAML: A smart contract language for distributed ledgers. <https://www.digitalasset.com/developers>, 2019.
 - [16] Phan Minh Dung. On the acceptability of arguments and its fundamental role in non-monotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995.
 - [17] European Parliament and Council. Regulation (EU) 2016/679 (general data protection regulation). Official Journal of the European Union L 119, 1–88. See Art. 22 and Recital 71, 2016.
 - [18] European Parliament and Council. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union L, 2024/1689, 2024.
 - [19] Edward A. Feigenbaum. Knowledge engineering: The applied side of artificial intelligence. *Annals of the New York Academy of Sciences*, 426:91–107, 1984.
 - [20] Paul Brilliant Feuto Njonko, Sylviane Cardey, Peter Greenfield, and Walid El Abed. RuleCNL: A controlled natural language for business rule specifications. In *Controlled Natural Language (CNL 2014)*, volume 8625 of *LNCS*, pages 66–77. Springer, 2014.
 - [21] Michael Gelfond and Vladimir Lifschitz. The stable model semantics for logic programming. In *Proceedings of the Fifth International Conference and Symposium on Logic*

- Programming (ICLP/SLP)*, pages 1070–1080. MIT Press, 1988.
- [22] Guido Governatori. Representing business contracts in RuleML. volume 14, pages 181–216, 2005.
 - [23] Guido Governatori, Francesco Olivieri, Antonino Rotolo, and Simone Scannapieco. Computing strong and weak permissions in defeasible logic. *Journal of Philosophical Logic*, 42(6):799–829, 2013.
 - [24] Michael Hanus. Functional logic programming: From theory to Curry. *Programming Logics: Essays in Memory of Harald Ganzinger, LNCS*, 7797:123–168, 2013.
 - [25] H. L. A. Hart. *The Concept of Law*. Oxford University Press, 1961.
 - [26] Bruce L. Hay and Kathryn E. Spier. Burdens of proof in civil litigation: An economic perspective. *The Journal of Legal Studies*, 26(2):413–431, 1997.
 - [27] Frederick Hayes-Roth, Donald A. Waterman, and Douglas B. Lenat, editors. *Building Expert Systems*. Addison-Wesley, 1983.
 - [28] C. A. R. Hoare. Communicating sequential processes. *Communications of the ACM*, 21(8):666–677, 1978.
 - [29] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice-Hall, Englewood Cliffs, NJ, 1985.
 - [30] Wesley Newcomb Hohfeld. Some fundamental legal conceptions as applied in judicial reasoning. *The Yale Law Journal*, 23(1):16–59, 1913.
 - [31] Gerard J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.
 - [32] Tom Hvitved. *Contract Formalisation and Modular Implementation of Domain-Specific Languages*. PhD thesis, Department of Computer Science, University of Copenhagen (DIKU), 2011.
 - [33] Tom Hvitved, Felix Klaedtke, and Eugen Zălinescu. A trace-based model for multiparty contracts. *The Journal of Logic and Algebraic Programming*, 81(2):72–98, 2012.
 - [34] Robert Kowalski and Akber Datto. Logical English for law and education. In *Prolog: The Next 50 Years, LNCS*, volume 13900, pages 287–299. Springer, 2023.
 - [35] Robert Kowalski and Marek Sergot. A logic-based calculus of events. *New Generation Computing*, 4(1):67–95, 1986.
 - [36] Tobias Kuhn. A survey and classification of controlled natural languages. *Computational Linguistics*, 40(1):121–170, 2014.
 - [37] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
 - [38] Peter J. Landin. The next 700 programming languages. *Communications of the ACM*, 9(3):157–166, 1966.
 - [39] Carl Malamud. Public.resource.org and law.gov: Making primary legal materials freely

- available. <https://public.resource.org/>; see also *Georgia v. Public.Resource.Org, Inc.*, 590 U.S. 255 (2020), 2020.
- [40] Denis Mériçoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. In *Proceedings of the ACM on Programming Languages (ICFP)*, volume 5, pages 1–29. ACM, 2021.
 - [41] James Mohun and Alex Roberts. Cracking the code: Rulemaking for humans and machines. OECD Working Papers on Public Governance No. 42, OECD Publishing, Paris, 2020.
 - [42] Object Management Group. Semantics of business vocabulary and business rules (SBVR), version 1.5. OMG Document formal/19-10-02, 2019.
 - [43] Object Management Group. Decision model and notation (DMN), version 1.3. OMG Document formal/21-02-01, 2021.
 - [44] Simon Peyton Jones, Jean-Marc Eber, and Julian Seward. Composing contracts: An adventure in financial engineering (functional pearl). In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 280–292. ACM, 2000.
 - [45] Henry Prakken and Giovanni Sartor. Law and logic: A review from an argumentation perspective. *Artificial Intelligence*, 227:214–245, 2015.
 - [46] Aarne Ranta. Grammatical framework: A type-theoretical grammar formalism. *Journal of Functional Programming*, 14(2):145–189, 2004.
 - [47] G. M. Reed and A. W. Roscoe. A timed model for communicating sequential processes. *Theoretical Computer Science*, 58(1–3):249–261, 1988.
 - [48] A. W. Roscoe. *The Theory and Practice of Concurrency*. Prentice-Hall, 1997.
 - [49] Ken Satoh, Kento Asai, Takamune Kogawa, Masahiro Kubota, Megumi Nakamura, Yoshiaki Nishigai, Kei Shirakawa, and Chiaki Takano. PROLEG: An implementation of the presupposed ultimate fact theory of Japanese civil code by PROLOG technology. In *New Frontiers in Artificial Intelligence (JSAI-isAI)*, volume 6797 of *LNCS*, pages 153–164. Springer, 2011.
 - [50] John R. Searle. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press, 1969.
 - [51] M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, and H. T. Cory. The british nationality act as a logic program. *Communications of the ACM*, 29(5):370–386, 1986.
 - [52] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of Mercury, an efficient purely declarative logic programming language. *The Journal of Logic Programming*, 29(1–3):17–64, 1996.
 - [53] Tarmo Uustalu and Varmo Vene. Comonadic notions of computation. *Electronic Notes*

- in Theoretical Computer Science*, 203(5):263–284, 2008.
- [54] Georg Henrik von Wright. Deontic logic. *Mind*, 60(237):1–15, 1951.
 - [55] Sandra Wachter, Brent Mittelstadt, and Luciano Floridi. Why a right to explanation of automated decision-making does not exist in the general data protection regulation. *International Data Privacy Law*, 7(2):76–99, 2017.
 - [56] Philip Wadler. The essence of functional programming. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 1–14. ACM, 1992.